# মেমোরি হায়ারার্কি (Memory Hierarchy)

লোকালিটি · RAM/ROM · ক্যাশ ম্যাপিং · VM/TLB · ডিস্ক: ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি




↑ পুরো টপিক এক পাতায়: লাল = হাই-ইল্ড, সবুজ = পলিসি / প্রয়োগ

## ➜ সিলেবাসের যে অংশ

"Computer Architecture: ... Processor design ... Pipeline ... Memory organization: cache, virtual memory, channels." । মাইক্রোপ্রসেসর পেপারে: "Memory Interfacing" । এই নোটে আছে হায়ারার্কি, সেমিকন্ডাক্টর মেমোরি, ধারণক্ষমতা ও ইন্টারলিভিং, ক্যাশ ডিজাইন এবং সেকেন্ডারি স্টোরেজ; CPU, পাইপলাইন ও CPI আছে CPU সংগঠন নোটে, পেজিং অ্যালগরিদম অপারেটিং সিস্টেম অধ্যায়ে।

**হাই-ইল্ড তালিকা**

① দ্রুততম = রেজিস্টার
② টেম্পোরাল বনাম স্পেশিয়াল
③ ক্যাশে SRAM, মেইনে DRAM
④ $T = h \cdot Tc + (1-h)(Tc+Tm)$
⑤ ট্যাগ/ইনডেক্স/অফসেট বিট
⑥ রাইট-ব্যাক + ডার্টি বিট
⑦ সিক + ল্যাটেন্সি + ট্রান্সফার

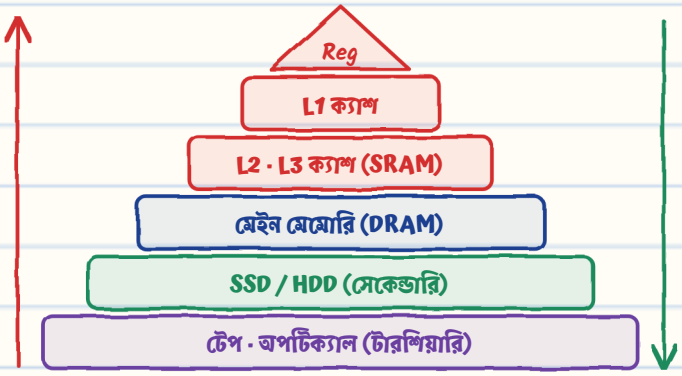
**সংকেত** হলুদ = মূল শব্দ · লাল বৃত্ত = মুখস্থ রাখার তথ্য · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

## ➜ সূচিপত্র

✓ হায়ারার্কি পিরামিড ও লোকালিটি ... পৃ. ২
✓ SRAM, DRAM ও ROM ..... পৃ. ৩
✓ ধারণক্ষমতা, চিপ, ইন্টারলিভিং ... পৃ. ৪
✓ ক্যাশ ও অ্যাক্সেস টাইম ..... পৃ. ৫
✓ ম্যাপিং: ট্যাগ / ইনডেক্স / অফসেট ... পৃ. ৬
✓ ম্যাপিং উদাহরণ, রিপ্লেসমেন্ট ... পৃ. ৭
✓ রাইট পলিসি, ৩টি C, মাল্টিলেভেল ... পৃ. ৮
✓ ভার্চুয়াল মেমোরি, TLB, ডিস্ক, RAID ... পৃ. ৯
✓ পরীক্ষার আগের রাতের পাতা ... পৃ. ১০
✓ অনুশীলন MCQ + লিখিত ..... পৃ. ১১

## ১ মেমোরি হায়ারার্কি কেন? MCQ + লিখিত

**সংজ্ঞা** মেমোরিকে কয়েকটি **স্তরে (level)** সাজানো হয়; প্রতিটি স্তর নিচেরটির চেয়ে ছোট, দ্রুত ও **প্রতি বিটে বেশি দামি**। লোকালিটির কারণে পুরো সিস্টেম চলে **প্রায় দ্রুততম স্তরের গতিতে, প্রায় সস্তাতম স্তরের খরচে**।




প্রাইমারি = যা CPU সরাসরি অ্যাক্সেস করে (ওপরের ৪টি)

লাল ↑ দ্রুত, বিটপ্রতি দামি · সবুজ ↓ বড়, সস্তা

**পিরামিডের নিচের দিকে** গেলে:
- ধারণক্ষমতা *(capacity)* ↑
- অ্যাক্সেস টাইম ↑ (ধীর)
- প্রতি বিটে খরচ ↓
- CPU-র অ্যাক্সেসের হার ↓

বৈশিষ্ট্য: **ইনক্লুশন *(inclusion)*** (স্তর $i \subset$ স্তর $i+1$), **কোহেরেন্স** (কপিগুলো মেলে), **লোকালিটি**। ৯০/১০ নিয়ম: সময়ের ≈ **৯০%** যায় কোডের ১০%-এ।

| স্তর | সাধারণ আকার | অ্যাক্সেস টাইম | উপাদান | নিয়ন্ত্রণ করে |
|---|---|---|---|---|
| রেজিস্টার | < 1 KB | **< 1 ns (1 সাইকেল)** | ফ্লিপ-ফ্লপ | কম্পাইলার |
| *L1 / L2 / L3* | *32 K / 1 M / 32 MB* | *1 / 4 / 15 ns* | *SRAM* | হার্ডওয়্যার |
| মেইন মেমোরি | *8 – 64 GB* | *50 – 100 ns* | *DRAM* | *OS* |
| *SSD / HDD* | *0.5 – 20 TB* | *0.1 ms / 5 – 10 ms* | ফ্ল্যাশ / ডিস্ক | *OS* |

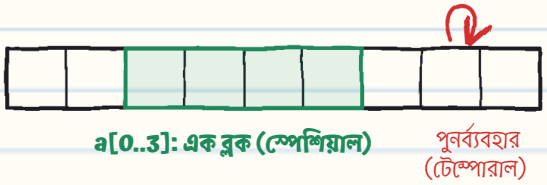
**পরীক্ষার ফাঁদ** দ্রুততম হলো **রেজিস্টার**, ক্যাশ নয়। *3 GHz-এ RAM ≈* **150 – 300 সাইকেল**।

## ২ লোকালিটি অব রেফারেন্স (Locality) হাই-ইল্ড

**টেম্পোরাল (কালিক)** এখন ব্যবহৃত আইটেম শিগগিরই আবার লাগে (লুপ কাউন্টার, *sum*, লুপের কোড)। সাম্প্রতিক আইটেম ক্যাশে **রেখে দিয়ে** এর সুবিধা নেওয়া হয় *(LRU)*।

**স্পেশিয়াল (স্থানিক)** কাছাকাছি অ্যাড্রেস শিগগিরই লাগে (অ্যারে, ক্রমিক কোড)। পুরো **ব্লক** *(64 B)* এনে ও প্রিফেচ করে এর সুবিধা নেওয়া হয়।

```
sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];    /* a[i]: স্পেশিয়াল */
                    /* sum, i: টেম্পোরাল */
```



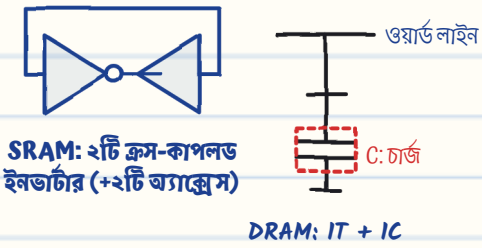

একটি মিসেই পাশের ডেটাও চলে আসে

**মনে রাখার কৌশল** *Temporal = same Thing, again in Time* (একই জিনিস আবার)। *Spatial = Surrounding Space* (আশপাশ)। C-তে অ্যারে row-major: *a[i][j]* লুপে **j ভেতরের লুপে** রাখো।

## ৩ RAM: SRAM বনাম DRAM

হাই-ইল্ড

| | SRAM | DRAM |
|---|---|---|
| সেল | 6T ফ্লিপ-ফ্লপ | 1T + 1C |
| রিফ্রেশ | লাগে না | প্রতি ≈ 64 ms |
| গতি | দ্রুত | ধীর |
| ঘনত্ব (density) | কম | বেশি |
| খরচ / বিট | বেশি | কম |
| রিড | নন-ডেস্ট্রাকটিভ | ডেস্ট্রাকটিভ |
| ব্যবহার | ক্যাশ | মেইন মেমোরি |




চার্জ লিক করে → রিফ্রেশ; "স্ট্যাটিক" = রিফ্রেশ নেই

**পরীক্ষার ফাঁদ** "স্ট্যাটিক" মানে নন-ভোলাটাইল **নয়**: বিদ্যুৎ গেলে SRAM ও DRAM **দুটিরই ডেটা মুছে যায়**। DRAM-এর রো ও কলাম অ্যাড্রেস একই পিন ভাগ করে (**RAS / CAS** দিয়ে মাল্টিপ্লেক্সড), তাই 1M × 1 DRAM-এ ২০টি নয়, মাত্র ১০টি অ্যাড্রেস পিন লাগে।

**DRAM পরিবার:** অ্যাসিনক্রোনাস DRAM → FPM → EDO → SDRAM (ক্লকড) → DDR (ক্লকের **দুই এজেই** ডেটা) → DDR2 / 3 / 4 / 5। সর্বোচ্চ হার = ট্রান্সফার/s × বাস প্রস্থ: 64-বিট (8 B) চ্যানেলে DDR4-3200 = $3200 \times 10^6 \times 8\ B$ = 25.6 GB/s।

↙ SRAM → ক্যাশ, DRAM → মেইন মেমোরি

## ৪ ROM পরিবার (নন-ভোলাটাইল)

| ধরন | লেখা হয় | মোছা হয় | সাধারণ ব্যবহার |
|---|---|---|---|
| Mask ROM | নির্মাতা (মাস্ক দিয়ে) | কখনো না | বিপুল উৎপাদনের কোড |
| PROM (OTP) | ব্যবহারকারী একবার (ফিউজ) | কখনো না | কম উৎপাদনের ফার্মওয়্যার |
| EPROM | ব্যবহারকারী, প্রোগ্রামারে | UV আলো পুরো চিপ | প্রোটোটাইপ (কোয়ার্টজ জানালা) |
| EEPROM | বৈদ্যুতিকভাবে, সার্কিটেই | বৈদ্যুতিকভাবে, **প্রতি বাইট** | সেটিংস, সিরিয়াল নম্বর |
| Flash | বৈদ্যুতিকভাবে, সার্কিটেই | বৈদ্যুতিকভাবে, **প্রতি ব্লক** | UEFI/BIOS, SSD, USB |

**NOR ফ্ল্যাশ:** র‍্যান্ডম বাইট রিড, এক্সিকিউট-ইন-প্লেস → ফার্মওয়্যার। **NAND ফ্ল্যাশ:** বেশি ঘন, পেজ রিড / ব্লক ইরেজ → SSD, মেমোরি কার্ড; ইরেজ সাইকেল সীমিত (ওয়্যার লেভেলিং)।

**মনে রাখার কৌশল** EPROM = **E**rase with **P**hotons (UV আলোয় মোছা)। EEPROM-এর বাড়তি E = **E**lectrically। ফ্ল্যাশ পুরো ব্লক মোছে "in a flash"।

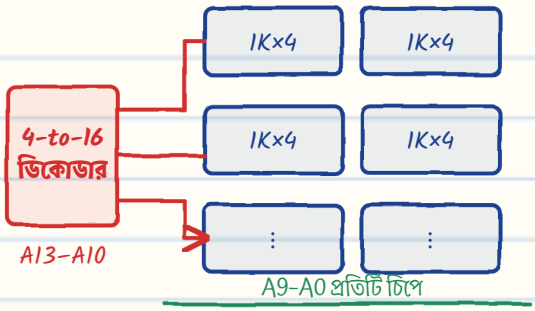
**পরীক্ষার ফাঁদ** ROM-ও **র‍্যান্ডম অ্যাক্সেস** ("RAM" আসলে রিড/রাইট মেমোরি বোঝায়); ROM প্রাইমারি, তবু নন-ভোলাটাইল। অ্যাক্সেস পদ্ধতি: **র‍্যান্ডম** (RAM, ROM) · **সিকোয়েনশিয়াল** (টেপ) · **ডাইরেক্ট** (ডিস্ক) · **অ্যাসোসিয়েটিভ**, কনটেন্ট দিয়ে (ক্যাশ ট্যাগ, TLB)।

## ৫ মেমোরি সংগঠন ও ধারণক্ষমতার হিসাব গাণিতিক

**মূল সূত্র** $2^n$ ওয়ার্ড × $m$ বিট → $n$ টি অ্যাড্রেস লাইন, $m$ টি ডেটা লাইন · চিপ সংখ্যা = (মোট ওয়ার্ড ÷ চিপের ওয়ার্ড) × (মোট প্রস্থ ÷ চিপের প্রস্থ) · অ্যাড্রেস বিট = $\log_2$(অ্যাড্রেসযোগ্য একক)

**প্রস্থ (ওয়ার্ড-সাইজ) বাড়ানো** চিপগুলো পাশাপাশি বসে **সব** অ্যাড্রেস লাইন ভাগ করে; প্রতিটি কিছু ডেটা বিট দেয় (দুটি 1K × 4 → 1K × 8)।

**গভীরতা (ওয়ার্ড-সংখ্যা) বাড়ানো** বাড়তি উচ্চ-ক্রমের অ্যাড্রেস বিট যায় একটি ডিকোডারে, যার আউটপুট চিপগুলোর **চিপ সিলেক্ট (CS)** চালায়।




1K × 4 দিয়ে 16K × 8: ১৬ সারি × ২টি চিপ

**সমাধানকৃত উদাহরণ**

১. একটি 4K × 8 চিপ: 4K = $2^{12}$ → **12**টি অ্যাড্রেস লাইন, **8**টি ডেটা লাইন; ধারণক্ষমতা 32 Kbit = 4 KB

২. 1K × 4 চিপ দিয়ে 16K × 8 মেমোরি: (16K / 1K) × (8 / 4) = 16 × 2 = **32টি চিপ**; 14টি অ্যাড্রেস লাইন: A9–A0 প্রতিটি চিপে, A13–A10 যায় **4-to-16 ডিকোডারে**

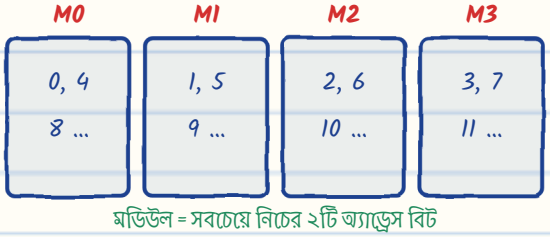
৩. 64 MB, বাইট-অ্যাড্রেসযোগ্য: $2^{26}$ বাইট → **26** অ্যাড্রেস বিট। একই মেমোরি, 32-বিট (4-বাইট) ওয়ার্ড, ওয়ার্ড-অ্যাড্রেসযোগ্য: $2^{24}$ ওয়ার্ড → **24** বিট

৪. 32-বিট বাইট অ্যাড্রেস → $2^{32}$ B = **4 GB**; 8086 (20টি লাইন) → 1 MB

**পরীক্ষার ফাঁদ** $\log_2$ নেওয়ার আগে দেখো মেমোরি **বাইট**- নাকি **ওয়ার্ড**-অ্যাড্রেসযোগ্য।

## ৬ মেমোরি ইন্টারলিভিং (Interleaving)

- **k টি মডিউল (ব্যাংক)** সমান্তরালে কাজ করে
- লো-অর্ডার: নিচের অ্যাড্রেস বিট মডিউল বাছে (address mod k); পরপর ওয়ার্ড **ভিন্ন** মডিউলে → পুরো ব্লক সমান্তরালে আসে
- **হাই-অর্ডার**: ওপরের বিট মডিউল বাছে; পরপর ওয়ার্ড **একই** মডিউলে → সহজে বাড়ানো যায়, গতি বাড়ে না



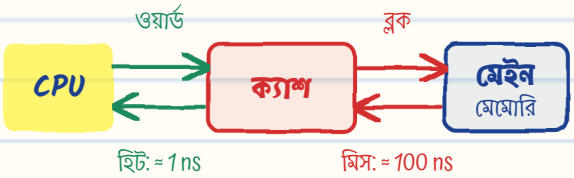

৪-ওয়ে লো-অর্ডার ইন্টারলিভিং

**সমাধানকৃত উদাহরণ** ৪-ওয়ার্ড ব্লক; অ্যাড্রেস পাঠাতে 1 সাইকেল, প্রতি DRAM অ্যাক্সেসে 15 সাইকেল, বাসে প্রতি ওয়ার্ডে 1 সাইকেল। এক-ওয়ার্ড-চওড়া মেমোরি: 1 + 4 × 15 + 4 × 1 = **65 সাইকেল**। ৪-ব্যাংক ইন্টারলিভড: 1 + 15 + 4 × 1 = **20 সাইকেল** (মিস পেনাল্টি 3.25 গুণ কম)।

## ৭ ক্যাশ মেমোরি: মূল কথা হাই-ইল্ড

**সংজ্ঞা** CPU ও মেইন মেমোরির মাঝে একটি ছোট, দ্রুত SRAM মেমোরি, যা সম্প্রতি ব্যবহৃত **ব্লকের** (লাইন) কপি রাখে। প্রোগ্রামারের কাছে অদৃশ্য: পুরোপুরি হার্ডওয়্যার চালায়।

- **ব্লক / লাইন**: ক্যাশ ও মেমোরির মধ্যে আদান-প্রদানের একক (সাধারণত 64 B)
- **হিট**: ওয়ার্ড ক্যাশে পাওয়া গেল · **মিস**: পাওয়া গেল না → ব্লক আনো
- হিট রেশিও $h$ = হিট / (হিট + মিস); মিস রেশিও = $1 - h$
- **হিট টাইম** ($T_c$) · **মিস পেনাল্টি** (ব্লক আনার সময়)
- প্রতি লাইনে থাকে **ট্যাগ** (কোন ব্লক আছে), **ভ্যালিড বিট**, আর রাইট-ব্যাকে **ডার্টি বিট**




CPU ↔ ক্যাশ: ওয়ার্ড · ক্যাশ ↔ RAM: ব্লক

## ৮ গড় অ্যাক্সেস টাইম: কোন সূত্র? খুব হাই-ইল্ড

| সংগঠন | একটি মিসের খরচ | সূত্র |
|---|---|---|
| **হায়ারার্কিক্যাল** (লুক-থ্রু) | $T_c + T_m$ | $T = h \cdot T_c + (1 - h)(T_c + T_m)$ |
| **সাইমালটেনিয়াস** (প্যারালাল) | শুধু $T_m$ | $T = h \cdot T_c + (1 - h) \cdot T_m$ |
| **AMAT** (Hennessy-Patterson) | মিস পেনাল্টি | AMAT = হিট টাইম + মিস রেট × পেনাল্টি |

হায়ারার্কিক্যাল: আগে ক্যাশ, তারপর মেমোরি। সাইমালটেনিয়াস: দুটিতে একসাথে খোঁজা শুরু। হায়ারার্কিক্যাল সূত্র সরল করলে $T_c + (1 - h) \cdot T_m$, অর্থাৎ পেনাল্টি = $T_m$ ধরলে AMAT-এর রূপ। প্রশ্নে কিছু না বললে **ডিফল্ট** হায়ারার্কিক্যাল, তবে অপশনে সাইমালটেনিয়াস উত্তরটিও আছে কি না দেখো।

**সমাধানকৃত উদাহরণ**

১. $T_c = 10$ ns, $T_m = 100$ ns, $h = 0.9$: হায়ারার্কিক্যাল $0.9 \times 10 + 0.1 \times 110 =$ **20 ns**; সাইমালটেনিয়াস $9 + 10 =$ **19 ns**; স্পিডআপ $= 100 / 20 =$ **5**

২. ক্লাসিক: $T_c = 100$ ns, $T_m = 1000$ ns, $h = 0.9$ → হায়ারার্কিক্যাল **200 ns** (সাইমালটেনিয়াস 190 ns)

৩. AMAT: হিট 1 ns, মিস রেট 5%, পেনাল্টি 100 ns → $1 + 0.05 \times 100 =$ **6 ns**

৪. $h$ নির্ণয়: $T_c = 5$, $T_m = 80$, লক্ষ্য 9 ns: $9 = 5 + (1 - h) \times 80$ → $h =$ **0.95**

৫. CPI (দেখো CPU সংগঠন): বেস 1, ইনস্ট্রাকশনপ্রতি 1.3 অ্যাক্সেস, মিস 2%, পেনাল্টি 100 → $1 + 1.3 \times 0.02 \times 100 =$ **3.6**

**মনে রাখার কৌশল** ক্যাশের "**ভেতর দিয়ে**" (through): মিসে **দুটোই** দিতে হয়, $T_c + T_m$। "**পাশাপাশি**" (side by side): মিসে শুধু $T_m$।

**পরীক্ষার ফাঁদ** **মিস** রেশিও (0.1) দেওয়া? সেটিকে $h$ ধরো না। যাচাই: $T_c \le T \le T_c + T_m$।

## ৯ ক্যাশ ম্যাপিং পদ্ধতি (Cache Mapping) খুব হাই-ইল্ড

ম্যাপিং ঠিক করে একটি মেমোরি ব্লক ক্যাশের **কোথায়** বসতে পারবে, তাই অ্যাড্রেস কীভাবে ভাগ হবে। ধরি ব্লক সাইজ = $B$ বাইট, লাইন = $L$, অ্যাসোসিয়েটিভিটি = $k$, সেট $S = L / k$।

**ডাইরেক্ট** ব্লক $j$ → লাইন $j \bmod L$। একটিই জায়গা, **একটি কম্পারেটর**। সরল, দ্রুত; **কনফ্লিক্ট মিস** (থ্র্যাশিং) বেশি।

**ফুলি অ্যাসোসিয়েটিভ** ব্লক → **যেকোনো** লাইনে; সব ট্যাগ **সমান্তরালে** তুলনা ($L$ টি কম্পারেটর, CAM)। কনফ্লিক্ট মিস নেই; ব্যয়বহুল।

**k-ওয়ে সেট অ্যাসো.** ব্লক $j$ → সেট $j \bmod S$, সেটের $k$ টি লাইনের যেকোনোটিতে; $k$ টি কম্পারেটর। বাস্তব CPU-তে ব্যবহৃত মধ্যপন্থা।

অফসেট = $\log_2 B$ · ট্যাগ = অ্যাড্রেস বিট − ইনডেক্স − অফসেট




**ডাইরেক্ট: লাইন = $\log_2 L$ বিট**




**ফুলি অ্যাসোসিয়েটিভ: ইনডেক্স ফিল্ড নেই**




**k-ওয়ে: সেট = $\log_2 (L / k)$ বিট**

তিন রকম অ্যাড্রেস বিভাজন (অ্যাসোসিয়েটিভিটি বাড়লে → ট্যাগ লম্বা, ইনডেক্স ছোট)

| | ডাইরেক্ট | k-ওয়ে সেট অ্যাসোসিয়েটিভ | ফুলি অ্যাসোসিয়েটিভ |
|---|---|---|---|
| ব্লক বসার জায়গা | 1 | $k$ | $L$ (যেকোনো) |
| ট্যাগ কম্পারেটর | 1 | $k$ | $L$ |
| কনফ্লিক্ট মিস | **সবচেয়ে বেশি** | কম | **নেই** |
| হার্ডওয়্যার খরচ, হিট টাইম | সবচেয়ে কম | মাঝারি | সবচেয়ে বেশি |
| রিপ্লেসমেন্ট পলিসি লাগে? | **না** (বিকল্প নেই) | হ্যাঁ, সেটের ভেতরে | হ্যাঁ |
| $k$-ওয়ের বিশেষ রূপ | $k = 1$ | $1 < k < L$ | $k = L$ (একটি সেট) |

### → একই ক্যাশ, ভিন্ন অ্যাসোসিয়েটিভিটি

| 64 KB · 64 B · 32-বিট | ট্যাগ | ইনডেক্স | অফসেট |
|---|---|---|---|
| ডাইরেক্ট (1024 লাইন) | **16** | 10 | 6 |
| 2-ওয়ে (512 সেট) | 17 | 9 | 6 |
| 4-ওয়ে (256 সেট) | 18 | 8 | 6 |
| 8-ওয়ে (128 সেট) | 19 | 7 | 6 |
| ফুলি অ্যাসোসিয়েটিভ | 26 | – | 6 |

**মনে রাখার কৌশল** $k$ **দ্বিগুণ** করলে ইনডেক্সের এক বিট ট্যাগে যায়। ডাইরেক্ট: ট্যাগ = $\log_2(\text{MM} \div \text{ক্যাশ}) = 32 - 16 = 16$।

**পরীক্ষার ফাঁদ** অফসেট ← **ব্লক** সাইজ; $k > 1$ হলে ইনডেক্স ← **সেট** সংখ্যা (লাইন নয়)।

**ব্লক 1000 কোথায় যাবে?** ডাইরেক্ট, 128 লাইন: $1000 \bmod 128$ = **লাইন 104**। 4-ওয়ে, 32 সেট: $1000 \bmod 32$ = **সেট 8** (এর ৪টি লাইনের যেকোনোটি)।

## ১০ ম্যাপিং: সমাধানকৃত উদাহরণ

লিখিত পরীক্ষায় প্রিয়

**উদাহরণ ১: অ্যাড্রেস ভাগ** ডাইরেক্ট-ম্যাপড 64 KB ক্যাশ, 64 B ব্লক, 32-বিট অ্যাড্রেস 0x12345678 (ট্যাগ 16 | লাইন 10 | অফসেট 6): অফসেট = 0x78 AND 0x3F = **56** · লাইন = (0x12345678 ÷ 64) mod 1024 = **345** · ট্যাগ = ওপরের 16 বিট = **0x1234**

**উদাহরণ ২: 2-ওয়ে সেট অ্যাসোসিয়েটিভ** 32 KB ক্যাশ, 32 B ব্লক, 32-বিট অ্যাড্রেস। অফসেট = $\log_2 32 = 5$; লাইন = 32 KB / 32 B = 1024; সেট = 1024 / 2 = 512 → সেট ফিল্ড = **9**; ট্যাগ = 32 − 9 − 5 = **18**।

**উদাহরণ ৩: ট্যাগ মেমোরি** উদাহরণ ১-এর ক্যাশ: 1024 লাইন × (16 ট্যাগ + 1 ভ্যালিড) = 17,408 বিট = **2176 B**, 64 KB ডেটার বাইরে বাড়তি।

### → উদাহরণ ৪: ৪-ব্লক ক্যাশে ব্লক ক্রম 0, 8, 0, 6, 8

| ক্যাশ | 0 | 8 | 0 | 6 | 8 | মিস |
|---|---|---|---|---|---|---|
| ডাইরেক্ট (j mod 4) | M | M | M | M | M | 5 |
| 2-ওয়ে, LRU | M | M | H | M | M | 4 |
| ফুলি অ্যাসো. | M | M | H | M | H | 3 |

ডাইরেক্ট: 0 ও 8 একই লাইন 0-তে, একে অপরকে সরায়। 2-ওয়ে: 0, 6, 8 তিনটিই সেট 0-তে (৩টি ব্লক, ২টি ওয়ে)। ফুলি অ্যাসো.: শুধু ৩টি কোল্ড মিস। ওয়ে বাড়ালে কনফ্লিক্ট মিস কমে।

## ১১ রিপ্লেসমেন্ট পলিসি (Replacement)

হাই-ইল্ড

কেবল তখনই লাগে, যখন ব্লক রাখার জন্য একাধিক লাইনের **বিকল্প** থাকে (সেট বা ফুলি অ্যাসোসিয়েটিভ) এবং সবগুলো ভরা।

| পলিসি | যে ব্লক সরায় … | মন্তব্য |
|---|---|---|
| **LRU** | সবচেয়ে বেশি সময় **অব্যবহৃত** | টেম্পোরাল লোকালিটি; 2-ওয়ে: প্রতি সেটে 1 বিট |
| **FIFO** | যেটি **সবার আগে** লোড হয়েছে | রাউন্ড-রবিন পয়েন্টার; **বেলাডির অ্যানোমালি** |
| **Random** | দৈবভাবে বাছাই করা | হিসাব রাখা লাগে না; বড় ক্যাশে প্রায় LRU-র সমান |
| LFU | সবচেয়ে কম বার ব্যবহৃত | কাউন্টার লাগে |
| Optimal (Belady) | ভবিষ্যতে সবচেয়ে দেরিতে লাগবে | ভবিষ্যৎ জানা লাগে: শুধু তুলনার মাপকাঠি |

**সমাধানকৃত ট্রেস** ফুলি অ্যাসোসিয়েটিভ, ৩টি লাইন (সাবস্ক্রিপ্টে সরানো ব্লক)

| রেফ | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | মিস |
|---|---|---|---|---|---|---|---|---|---|---|---|
| FIFO | M | M | M | $M_7$ | H | $M_0$ | $M_1$ | $M_2$ | $M_3$ | $M_0$ | 9 |
| LRU | M | M | M | $M_7$ | H | $M_1$ | H | $M_2$ | $M_3$ | $M_0$ | 8 |
| OPT | M | M | M | $M_7$ | H | $M_1$ | H | $M_0$ | H | H | 6 |

**মনে রাখার কৌশল** FIFO: **জন্মতারিখ** (কে আগে এল) · LRU: **শেষ ব্যবহার** · OPT: **পরের ব্যবহার** (পেজিংয়েও একই অ্যালগরিদম)।

## ১২ রাইট পলিসি (Write Policy) হাই-ইল্ড

| রাইট হিটে | রাইট-থ্রু | রাইট-ব্যাক (কপি-ব্যাক) |
|---|---|---|
| মেমোরি হালনাগাদ | **প্রতিটি** রাইটে | **ডার্টি** ব্লক সরানোর সময় |
| মেমোরি ট্রাফিক | বেশি (রাইট বাফার আড়াল করে) | কম |
| মেমোরি হালনাগাদ থাকে? | সবসময় (DMA-র জন্য নিরাপদ) | না: ডার্টি বিট, কোহেরেন্স সামলাতে হয় |
| রাইট মিসে সাধারণ জুটি | **নো-রাইট-অ্যালোকেট** | **রাইট-অ্যালোকেট** |

রাইট-অ্যালোকেট (*fetch-on-write*): রাইট মিসে আগে ব্লক লোড, তারপর লেখা। **নো-রাইট-অ্যালোকেট** (*write-around*): সরাসরি মেমোরিতে লেখা।

**ট্রাফিক ✍️** একটি লুপ একটি ভেরিয়েবলে ১০০ বার লেখে: রাইট-থ্রু **100**টি মেমোরি রাইট, রাইট-ব্যাক **1**টি (সরানোর সময়)।

**রাইট-ব্যাক টাইম ✍️** $Tc = 5$ ns, $Tm = 50$ ns, $h = 0.9$, সরানো ব্লকের 30% ডার্টি: $T = 5 + 0.1 \times (50 + 0.3 \times 50) =$ **11.5 ns**

মাল্টিকোর: একাধিক ক্যাশে একই ব্লকের কপি থাকতে পারে; একটি **কোহেরেন্স প্রোটোকল** (*MESI*, স্নুপিং) সেগুলো মিলিয়ে রাখে।

## ১৩ মিস কেন হয়: ৩টি C MCQ

| মিস | কারণ | প্রতিকার |
|---|---|---|
| **কম্পালসরি** | প্রথম অ্যাক্সেস (কোল্ড) | বড় ব্লক, প্রিফেচ |
| **ক্যাপাসিটি** | ওয়ার্কিং সেট > ক্যাশ | বড় ক্যাশ |
| **কনফ্লিক্ট** | সেটে ব্লকের সংঘর্ষ | বেশি ওয়ে, ভিক্টিম ক্যাশ |
| কোহেরেন্স | অন্য কোর লিখেছে | (৪র্থ C) |




ব্লক সাইজ: U-আকৃতির ট্রেড-অফ

**মনে রাখার কৌশল** **C**old start, **C**rowded cache, **C**lash in a set (ঠান্ডা শুরু, ভিড়, সংঘর্ষ)। ফুলি অ্যাসোসিয়েটিভে কনফ্লিক্ট মিস **নেই**।

**পরীক্ষার ফাঁদ** বড় ব্লক কোল্ড মিস কমায়, কিন্তু **মিস পেনাল্টি** বাড়ায়।

## ১৪ মাল্টিলেভেল ক্যাশ

- **L1**: আলাদা I + D ক্যাশ, প্রতি কোরে; লক্ষ্য: কম হিট টাইম
- **L2**: ইউনিফায়েড, প্রতি কোরে; লক্ষ্য: কম মিস রেট
- **L3**: সব কোরের শেয়ার্ড (লাস্ট লেভেল)
- **লোকাল** মিস রেট = মিস / **ওই স্তরে** অ্যাক্সেস; **গ্লোবাল** = মিস / CPU-র মোট অ্যাক্সেস

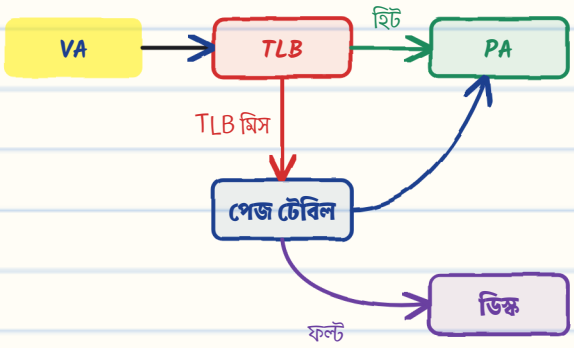
**দুই-স্তর AMAT ✍️** L1: 1 ns, মিস 5%। L2: 10 ns, লোকাল মিস 20%। RAM 100 ns।
AMAT = $1 + 0.05 \times (10 + 0.2 \times 100) = 1 + 0.05 \times 30 =$ **2.5 ns**
L2 ছাড়া: $1 + 0.05 \times 100 = 6$ ns। গ্লোবাল L2 মিস রেট = $0.05 \times 0.2 =$ **1%**।

## ১৫ ভার্চুয়াল মেমোরি ও TLB (সারসংক্ষেপ) MCQ

VM মেইন মেমোরিকে ডিস্কের ক্যাশ হিসেবে ব্যবহার করে। **MMU** পেজ ধরে ধরে **পেজ টেবিলের** সাহায্যে ভার্চুয়াল অ্যাড্রেস অনুবাদ করে (টেবিল RAM-এ, PTBR দিয়ে খোঁজা হয়)।

পেজ ফল্টের খরচ **লক্ষ লক্ষ সাইকেল** → বড় পেজ (4 KB), ফুলি অ্যাসোসিয়েটিভ প্লেসমেন্ট, LRU-ধাঁচের রিপ্লেসমেন্ট, সবসময় **রাইট-ব্যাক**।

TLB = সাম্প্রতিক পেজ-টেবিল এন্ট্রির ছোট অ্যাসোসিয়েটিভ ক্যাশ।




| ক্যাশ | VM | সূত্র / উদাহরণ |
|---|---|---|
| ব্লক | পেজ / ফ্রেম | পেজ সংখ্যা = 2^(VA বিট − অফসেট বিট) |
| মিস | পেজ ফল্ট (OS) | 32-বিট VA, 4 KB পেজ → $2^{20}$ PTE × 4 B = **4 MB** |
| ট্যাগ স্টোর | পেজ টেবিল, TLB | 64 MB RAM, 4 KB পেজ → $2^{14}$ ফ্রেম |

**TLB-সহ EAT** ✍️ TLB 20 ns, মেমোরি 100 ns, TLB হিট 0.8, এক-স্তর টেবিল: 0.8 × (20 + 100) + 0.2 × (20 + 100 + 100) = **140 ns** (TLB মিসে একটি বাড়তি মেমোরি অ্যাক্সেস)। পেজিং অ্যালগরিদম, থ্র্যাশিং → OS অধ্যায়।

## ১৬ সেকেন্ডারি স্টোরেজ: ডিস্ক ও RAID গাণিতিক

**ডিস্কের সূত্র** অ্যাক্সেস টাইম = সিক + রোটেশনাল ল্যাটেন্সি + ট্রান্সফার (+ কন্ট্রোলার ওভারহেড) · গড় ল্যাটেন্সি = ½ × (60 / rpm) s · ধারণক্ষমতা = সারফেস × ট্র্যাক × সেক্টর/ট্র্যাক × বাইট/সেক্টর

**সমাধানকৃত উদাহরণ** ✍️ 6000 rpm, গড় সিক 8 ms, প্রতি ট্র্যাকে 100 সেক্টর × 512 B।

১. এক ঘূর্ণন = 60 / 6000 s = 10 ms → ল্যাটেন্সি **5 ms**; 1 সেক্টর = 10 / 100 = 0.1 ms

২. অ্যাক্সেস = 8 + 5 + 0.1 = **13.1 ms**; 4 KB (8 সেক্টর) = 13.8 ms; হার = 51,200 B / 10 ms = **5.12 MB/s**

৩. 16 সারফেস × 1024 ট্র্যাক × 128 সেক্টর × 512 B = $2^{30}$ B = **1 GB**

| RAID | পদ্ধতি | ন্যূনতম | ব্যবহারযোগ্য | নষ্ট সহ্য করে |
|---|---|---|---|---|
| 0 | স্ট্রাইপিং, রিডানডেন্সি নেই | 2 | n | **একটিও না** |
| 1 | মিররিং | 2 | n / 2 | প্রতি জোড়ায় 1টি |
| 5 | স্ট্রাইপিং + **ডিস্ট্রিবিউটেড প্যারিটি** | 3 | n − 1 | 1টি ডিস্ক |
| 6 | দুটি ডিস্ট্রিবিউটেড প্যারিটি | 4 | n − 2 | 2টি ডিস্ক |
| 10 | মিরর, তারপর স্ট্রাইপ | 4 | n / 2 | প্রতি জোড়ায় 1টি |

RAID 2 = বিট-লেভেল হ্যামিং, 3 = বাইট-লেভেল, 4 = ব্লক-লেভেল; প্রতিটিতে একটি **আলাদা (ডেডিকেটেড)** প্যারিটি ডিস্ক (বটলনেক)। প্যারিটি = XOR। 5 × 2 TB: RAID 5 → **8 TB**, RAID 6 → 6 TB।

# পরীক্ষার আগের রাতের পাতা

## তথ্য-তালিকা

| | |
|---|---|
| Reg → L1 → L2 → L3 → RAM → ডিস্ক → টেপ | নিচে: আকার ↑, সময় ↑, খরচ/বিট ↓ |
| টেম্পোরাল = একই আইটেম · স্পেশিয়াল = পাশের | SRAM 6T, ক্যাশ · DRAM 1TIC, রিফ্রেশ |
| EPROM: UV · EEPROM: বাইট · ফ্ল্যাশ: ব্লক | $2^n \times m$ → $n$ অ্যাড্রেস, $m$ ডেটা লাইন |
| চিপ = (W/w) × (M/m) | লো-অর্ডার: মডিউল = addr mod k |
| হায়ারা.: $T = h \cdot Tc + (1-h)(Tc+Tm)$ | সাইমাল.: $T = h \cdot Tc + (1-h) \cdot Tm$ |
| AMAT = হিট টাইম + মিস রেট × পেনাল্টি | অফসেট = $\log_2 B$ · সেট = $L / k$ |
| ট্যাগ = অ্যাড্রেস − ইনডেক্স − অফসেট | ডাইরেক্ট ট্যাগ = $\log_2$(MM / ক্যাশ) |
| EAT(TLB) = h(t+m) + (1−h)(t+2m) | ডিস্ক = সিক + ½ ঘূর্ণন + ট্রান্সফার |

## → MCQ-র জন্য সেরা ১২টি এক-লাইনার

1. দ্রুততম স্টোরেজ: CPU রেজিস্টার
2. ক্যাশ থাকে CPU ও মেইন মেমোরির মাঝে
3. ক্যাশ তৈরি হয় SRAM দিয়ে
4. DRAM-এ রিফ্রেশ লাগে; SRAM-এ লাগে না
5. লুপ ভেরিয়েবল বারবার = টেম্পোরাল লোকালিটি
6. যেকোনো ব্লক → যেকোনো লাইন = ফুলি অ্যাসো.
7. ডাইরেক্ট ম্যাপিংয়ে রিপ্লেসমেন্ট পলিসি লাগে না
8. ডার্টি বিট রাইট-ব্যাকের অংশ
9. FIFO-তে বেলাডির অ্যানোমালি হতে পারে
10. TLB = পেজ-টেবিল এন্ট্রির ক্যাশ
11. RAID 5 = ডিস্ট্রিবিউটেড প্যারিটি, $n - 1$
12. 7200 rpm → গড় ল্যাটেন্সি 4.17 ms

**প্রধান ফাঁদ** ক্যাশ নয়, রেজিস্টার দ্রুততম · SRAM-ও ভোলাটাইল · হায়ারার্কিক্যাল (মিসে Tc + Tm) বনাম সাইমালটেনিয়াস (Tm) · $k > 1$ হলে ইনডেক্স গোনে **সেট** · বাইট বনাম ওয়ার্ড অ্যাড্রেসিং · ল্যাটেন্সি = **অর্ধেক** ঘূর্ণন · RAID 0-তে রিডানডেন্সি নেই · বড় ব্লক সবসময় ভালো নয়

**সব কৌশল একসাথে** Temporal = same Thing in Time, Spatial = Surrounding Space · EPROM = Erase with Photons · "ভেতর দিয়ে" Tc + Tm, "পাশাপাশি" শুধু Tm · k দ্বিগুণ → ইনডেক্সের এক বিট ট্যাগে · FIFO জন্মতারিখ, LRU শেষ ব্যবহার, OPT পরের ব্যবহার · ৩টি C: Cold, Crowded, Clash




১. অফসেট = $\log_2$ ব্লক
২. ইনডেক্স = $\log_2$ (লাইন / k)
৩. ট্যাগ = বাকি বিট

৩ ধাপে অ্যাড্রেস বিভাজন: সবসময় আগে অফসেট, শেষে ট্যাগ

## ✓ অনুশীলন MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

আগে নিজে করো, পরে মেলাও

**১.** কোন মেমোরিতে নিয়মিত রিফ্রেশ লাগে?
ক) SRAM খ) DRAM
গ) EPROM ঘ) ফ্ল্যাশ

**২.** হায়ারার্কিক্যাল: Tc 20 ns, Tm 200 ns, h 0.9। T = ?
ক) 38 ns খ) 36 ns
গ) 40 ns ঘ) 22 ns

**৩.** 1K × 4 চিপ দিয়ে 16K × 8 বানাতে লাগে:
ক) 32টি চিপ খ) 16টি চিপ
গ) 8টি চিপ ঘ) 64টি চিপ

**৪.** ডাইরেক্ট, 32 KB, 16 B ব্লক, 32-বিট: ট্যাগ বিট?
ক) 15 খ) 21
গ) 11 ঘ) 17

**৫.** ডার্টি বিট দরকার হয়:
ক) রাইট-ব্যাকে খ) রাইট-থ্রুতে
গ) ডাইরেক্ট ম্যাপিংয়ে ঘ) FIFO-তে

**৬.** দুটি ব্লক একই লাইনের জন্য লড়লে হয়:
ক) কম্পালসরি মিস খ) ক্যাপাসিটি মিস
গ) কনফ্লিক্ট মিস ঘ) কোহেরেন্স মিস

**৭.** ক্রম অনুযায়ী অ্যারে স্ক্যান কাজে লাগায়:
ক) টেম্পোরাল লোকালিটি খ) স্পেশিয়াল লোকালিটি
গ) ইন্টারলিভিং ঘ) রাইট-অ্যালোকেট

**৮.** 7200 rpm ডিস্কের গড় ল্যাটেন্সি:
ক) 8.33 ms খ) 7.2 ms
গ) 2.08 ms ঘ) 4.17 ms

**৯.** ডিস্ট্রিবিউটেড প্যারিটিসহ স্ট্রাইপিং হলো:
ক) RAID 5 খ) RAID 1
গ) RAID 4 ঘ) RAID 0

**১০.** k = লাইনের সংখ্যা হলে k-ওয়ে ক্যাশটি:
ক) ডাইরেক্ট ম্যাপড খ) 2-ওয়ে
গ) ফুলি অ্যাসোসিয়েটিভ ঘ) ইন্টারলিভড

উত্তরমালা → ১-খ · ২-গ · ৩-ক · ৪-ঘ · ৫-ক · ৬-গ · ৭-খ · ৮-ঘ · ৯-ক · ১০-গ

**লিখিত প্রশ্ন** তিন ধরনের ক্যাশ ম্যাপিং ব্যাখ্যা করো। 64 B ব্লক ও 32-বিট অ্যাড্রেসের একটি 256 KB, 4-ওয়ে ক্যাশের ফিল্ডগুলোর প্রস্থ, 0x0001F2C4 অ্যাড্রেসের সেট ও ট্যাগ, এবং ট্যাগ স্টোরেজ (প্রতি লাইনে ১টি ভ্যালিড বিট) নির্ণয় করো। ৮ নম্বর

১ ডাইরেক্ট: লাইন = j mod L · ফুলি অ্যাসোসিয়েটিভ: যেকোনো লাইন · k-ওয়ে: সেট = j mod S (প্রতিটির একটি সুবিধা / অসুবিধা) ৩

২ অফসেট = $\log_2 64$ = **6**; 4096 লাইন ÷ 4 = 1024 সেট → **10**; ট্যাগ = 32 − 10 − 6 = **16** ২

৩ অফসেট = 0xC4 AND 0x3F = **4**; সেট = ⌊0x1F2C4 ÷ 64⌋ mod 1024 = 1995 mod 1024 = **971**; ট্যাগ = **0x0001** ২

৪ ট্যাগ স্টোরেজ = 4096 × (16 + 1) = 69,632 বিট = **8.5 KB** ১

### → পরীক্ষার আগে যাচাই ✓

- ✓ কোন অ্যাক্সেস-টাইম সূত্র কখন খাটে, জানি
- ✓ যেকোনো অ্যাড্রেস ট্যাগ / ইনডেক্স / অফসেটে ভাগ করতে পারি
- ✓ ব্লক ক্রমে FIFO ও LRU ট্রেস করতে পারি
- ✓ ডিস্ক অ্যাক্সেস টাইম ও RAID ধারণক্ষমতা হিসাব করতে পারি

**লিখিতের জন্য আরও অনুশীলন:**
① SRAM বনাম DRAM, ৬টি পার্থক্য
② 4K × 8 চিপ দিয়ে 32K × 16
③ রাইট-থ্রু বনাম রাইট-ব্যাক
④ গ্লোবাল মিস রেটসহ দুই-স্তর AMAT